

DAA Practicals

Vansh Garg
Bsc (Hons) Computer Science
25821

Q1) Write a program to sort the elements of an array using Insertion Sort (The program should report the number of comparisons).

```
1  #include <iostream>
2  using namespace std;
3
4  void insertionSort(int arr[], int n)
5  {
6      int comparisons = 0;
7
8      for (int i = 1; i < n; i++)
9      {
10         int key = arr[i];
11         int j = i - 1;
12
13         while (j >= 0 && arr[j] > key)
14         {
15             comparisons++;
16             arr[j + 1] = arr[j];
17             j--;
18         }
19
20         if (j >= 0)
21             comparisons++;
22
23         arr[j + 1] = key;
24     }
25
26     cout << "Sorted Array:\n";
27
28     for (int i = 0; i < n; i++)
29         cout << arr[i] << " ";
30
31     cout << "\nComparisons = " << comparisons;
32 }
33
34 int main()
35 {
36     int arr[] = {5, 2, 4, 1, 3};
37     int n = 5;
38
39     insertionSort(arr, n);
40 }
```

```
DAA/prac □ main
> g++ insertion.cpp -o insertion

DAA/prac □ main ?
> ./insertion
Sorted Array:
1 2 3 4 5
Comparisons = 9%
```

Q2) Write a program to sort the elements of an array using Merge Sort (The program should report the number of comparisons).

```
1  #include <iostream>
2  using namespace std;
3
4  int comparisons = 0;
5
6  void merge(int arr[], int l, int m, int r)
7  {
8      int n1 = m - l + 1;
9      int n2 = r - m;
10
11      int L[n1], R[n2];
12
13      for (int i = 0; i < n1; i++)
14          L[i] = arr[l + i];
15
16      for (int j = 0; j < n2; j++)
17          R[j] = arr[m + 1 + j];
18
19      int i = 0, j = 0, k = l;
20
21      while (i < n1 && j < n2)
22      {
23          comparisons++;
24
25          if (L[i] <= R[j])
26              arr[k++] = L[i++];
27          else
28              arr[k++] = R[j++];
29      }
30
31      while (i < n1)
32          arr[k++] = L[i++];
33
34      while (j < n2)
35          arr[k++] = R[j++];
36  }
37
38  void mergeSort(int arr[], int l, int r)
39  {
40      if (l < r)
41      {
42          int m = (l + r) / 2;
43
44          mergeSort(arr, l, m);
45          mergeSort(arr, m + 1, r);
46
47          merge(arr, l, m, r);
48      }
49  }
50
51  int main()
52  {
53      int arr[] = {5, 2, 4, 1, 3};
54      int n = 5;
55
56      mergeSort(arr, 0, n - 1);
57
58      for (int i = 0; i < n; i++)
59          cout << arr[i] << " ";
60
61      cout << "\nComparisons = " << comparisons;
62  }
```

```
DAA/prac □ main ?
> g++ merge.cpp -o merge

DAA/prac □ main ?
> ./merge
1 2 3 4 5
Comparisons = 7%
```

Q3) Write a program to sort the elements of an array using Heap Sort (The program should report the number of comparisons).

```
1  #include <iostream>
2  using namespace std;
3
4  int comparisons = 0;
5
6  void heapify(int arr[], int n, int i)
7  {
8      int largest = i;
9
10     int left = 2 * i + 1;
11     int right = 2 * i + 2;
12
13     if (left < n)
14     {
15         comparisons++;
16
17         if (arr[left] > arr[largest])
18         {
19             largest = left;
20         }
21     }
22
23     if (right < n)
24     {
25         comparisons++;
26
27         if (arr[right] > arr[largest])
28         {
29             largest = right;
30         }
31     }
32
33     if (largest != i)
34     {
35         swap(arr[i], arr[largest]);
36         heapify(arr, n, largest);
37     }
38 }
39
40 void heapSort(int arr[], int n)
41 {
42     for (int i = n / 2 - 1; i >= 0; i--)
43     {
44         heapify(arr, n, i);
45     }
46
47     for (int i = n - 1; i > 0; i--)
48     {
49         // Move largest element to end
50         swap(arr[0], arr[i]);
51
52         // Heapify reduced heap
53         heapify(arr, i, 0);
54     }
55 }
56
57 int main()
58 {
59     int arr[] = {5, 2, 8, 1, 3};
60     int n = 5;
61
62     heapSort(arr, n);
63
64     cout << "Sorted Array:\n";
65
66     for (int i = 0; i < n; i++)
67     {
68         cout << arr[i] << " ";
69     }
70
71     cout << "\nComparisons = " << comparisons;
72
73     return 0;
74 }
```

```
DAA/prac □ main ?
➤ g++ heap.cpp -o heap
```

```
DAA/prac □ main ?
➤ ./heap
```

Sorted Array:

1 2 3 5 8

Comparisons = 9%

Q4) Write a program to sort the elements of an array using Quick Sort (The program should report the number of comparisons).

```
1  #include <iostream>
2  using namespace std;
3
4  int comparisons = 0;
5
6  int partition(int arr[], int low, int high)
7  {
8      int pivot = arr[high]; // last element as pivot
9      int i = low - 1;
10
11     for (int j = low; j < high; j++)
12     {
13         comparisons++;
14
15         if (arr[j] < pivot)
16         {
17             i++;
18             int temp = arr[i];
19             arr[i] = arr[j];
20             arr[j] = temp;
21         }
22     }
23
24     // place pivot in correct position
25     int temp = arr[i + 1];
26     arr[i + 1] = arr[high];
27     arr[high] = temp;
28
29     return i + 1;
30 }
31
32 // Quick Sort function
33 void quickSort(int arr[], int low, int high)
34 {
35     if (low < high)
36     {
37         int pi = partition(arr, low, high);
38
39         quickSort(arr, low, pi - 1);
40
41         quickSort(arr, pi + 1, high);
42     }
43 }
44
45 int main()
46 {
47     int arr[] = {5, 2, 8, 1, 3};
48
49     int n = 5;
50
51     quickSort(arr, 0, n - 1);
52
53     cout << "Sorted Array:\n";
54
55     for (int i = 0; i < n; i++)
56         cout << arr[i] << " ";
57
58     cout << "\nComparisons = " << comparisons;
59
60     return 0;
61 }
```

```
DAA/prac □ main ?
➤ g++ quick.cpp -o quick

DAA/prac □ main ?
➤ ./quick
Sorted Array:
1 2 3 5 8
Comparisons = 6%
```

Q5) Write a program to multiply two matrices using the Strassen's algorithm for matrix multiplication.

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int A[2][2] = {{1, 2}, {3, 4}};
7      int B[2][2] = {{5, 6}, {7, 8}};
8      int C[2][2];
9
10     int p1 = A[0][0] * (B[0][1] - B[1][1]);
11     int p2 = (A[0][0] + A[0][1]) * B[1][1];
12     int p3 = (A[1][0] + A[1][1]) * B[0][0];
13     int p4 = A[1][1] * (B[1][0] - B[0][0]);
14     int p5 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
15     int p6 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);
16     int p7 = (A[0][0] - A[1][0]) * (B[0][0] + B[0][1]);
17
18     C[0][0] = p5 + p4 - p2 + p6;
19     C[0][1] = p1 + p2;
20     C[1][0] = p3 + p4;
21     C[1][1] = p1 + p5 - p3 - p7;
22
23     cout << "Result Matrix:\n";
24
25     for (int i = 0; i < 2; i++)
26     {
27         for (int j = 0; j < 2; j++)
28             cout << C[i][j] << " ";
29
30         cout << endl;
31     }
32 }
```

```
DAA/prac □ main ?
➤ g++ matrix.cpp -o matrix
```

```
DAA/prac □ main ?
➤ ./matrix
```

Result Matrix:

19 22

43 50

Q6) Write a program to sort the elements of an array using Count Sort..

```
1  #include <iostream>
2  using namespace std;
3
4  void countSort(int arr[], int n)
5  {
6      int comparisons = 0;
7
8      // Find maximum element
9      int max = arr[0];
10
11     for (int i = 1; i < n; i++)
12     {
13         comparisons++;
14
15         if (arr[i] > max)
16         {
17             max = arr[i];
18         }
19     }
20
21     // Create count array
22     int count[max + 1] = {0};
23
24     // Store frequency of elements
25     for (int i = 0; i < n; i++)
26     {
27         count[arr[i]]++;
28     }
29
30     // Build sorted array
31     int index = 0;
32
33     for (int i = 0; i <= max; i++)
34     {
35         while (count[i] > 0)
36         {
37             arr[index] = i;
38             index++;
39             count[i]--;
40         }
41     }
42
43     // Print sorted array
44     cout << "Sorted Array:\n";
45
46     for (int i = 0; i < n; i++)
47     {
48         cout << arr[i] << " ";
49     }
50
51     cout << "\nComparisons = " << comparisons;
52 }
53
54 int main()
55 {
56     int arr[] = {4, 2, 2, 8, 3, 3, 1};
57     int n = 7;
58
59     countSort(arr, n);
60
61     return 0;
62 }
```

```
DAA/prac 0 main ?
➤ g++ count.cpp -o count
```

```
DAA/prac 0 main ?
➤ ./count
```

Sorted Array:

1 2 2 3 3 4 8

Comparisons = 6%

Q7) Display the data stored in a given graph using the Breadth-First Search algorithm.

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  int main()
6  {
7      // Graph with 5 vertices
8      int graph[5][5] = {
9          {0, 1, 1, 0, 0},
10         {1, 0, 0, 1, 1},
11         {1, 0, 0, 0, 0},
12         {0, 1, 0, 0, 0},
13         {0, 1, 0, 0, 0}};
14
15     bool visited[5] = {false};
16
17     queue<int> q;
18
19     // Start BFS from node 0
20     visited[0] = true;
21     q.push(0);
22
23     cout << "BFS Traversal: ";
24
25     while (!q.empty())
26     {
27         int node = q.front();
28         q.pop();
29
30         cout << node << " ";
31
32         // Check connected nodes
33         for (int i = 0; i < 5; i++)
34         {
35             if (graph[node][i] == 1 && !visited[i])
36             {
37                 visited[i] = true;
38                 q.push(i);
39             }
40         }
41     }
42
43     return 0;
44 }
```

```
DAA/prac □ main ?
> g++ BFS.cpp -o BFS

DAA/prac □ main ?
> ./BFS
BFS Traversal: 0 1 2 3 4 %
```

Q8) Display the data stored in a given graph using the Depth-First Search algorithm.

```
1  #include <iostream>
2  using namespace std;
3
4  void DFS(int graph[5][5], bool visited[], int node)
5  {
6      visited[node] = true;
7
8      cout << node << " ";
9
10     for (int i = 0; i < 5; i++)
11     {
12         if (graph[node][i] == 1 && !visited[i])
13         {
14             DFS(graph, visited, i);
15         }
16     }
17 }
18
19 int main()
20 {
21     int graph[5][5] =
22     {
23         {0, 1, 1, 0, 0},
24         {1, 0, 0, 1, 1},
25         {1, 0, 0, 0, 0},
26         {0, 1, 0, 0, 0},
27         {0, 1, 0, 0, 0}};
28
29     bool visited[5] = {false};
30
31     cout << "DFS Traversal: ";
32
33     DFS(graph, visited, 0);
34
35     return 0;
36 }
```

```
DAA/prac □ main ?
> g++ DFS.cpp -o DFS

DAA/prac □ main ?
> ./DFS
DFS Traversal: 0 1 3 4 2 %
```


Q9) Write a program to determine a minimum spanning tree of a graph using the Prim's algorithm.

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      // Adjacency matrix of graph
7      int graph[4][4] = {
8          {0, 2, 0, 6},
9          {2, 0, 3, 8},
10         {0, 3, 0, 0},
11         {6, 8, 0, 0}};
12
13     // Keeps track of selected vertices
14     bool selected[4] = {false};
15
16     // Start from vertex 0
17     selected[0] = true;
18
19     int edges = 0;
20
21     cout << "Edges in Minimum Spanning Tree:\n";
22
23     // MST will have (vertices - 1) edges
24     while (edges < 3)
25     {
26         int min = 999;
27         int x = 0;
28         int y = 0;
29
30         // Find minimum edge
31         for (int i = 0; i < 4; i++)
32         {
33             if (selected[i])
34             {
35                 for (int j = 0; j < 4; j++)
36                 {
37                     // If vertex not selected and edge exists
38                     if (!selected[j] && graph[i][j])
39                     {
40                         if (graph[i][j] < min)
41                         {
42                             min = graph[i][j];
43                             x = i;
44                             y = j;
45                         }
46                     }
47                 }
48             }
49         }
50
51         cout << x << " - " << y
52              << " : " << graph[x][y] << endl;
53
54         // Include new vertex in MST
55         selected[y] = true;
56
57         edges++;
58     }
59
60     return 0;
61 }
```

```
DAA/prac 0 main ?
> g++ prims.cpp -o prims
```

```
DAA/prac 0 main ?
> ./prims
Edges in Minimum Spanning Tree:
0 - 1 : 2
1 - 2 : 3
0 - 3 : 6
```

Q10) Write a program to solve the 0-1 knapsack problem.

```
1  #include <iostream>
2  using namespace std;
3
4  int max(int a, int b)
5  {
6      if (a > b)
7          return a;
8      else
9          return b;
10 }
11
12 int knapsack(int W, int wt[], int val[], int n)
13 {
14     // Base condition
15     if (n == 0 || W == 0)
16         return 0;
17
18     if (wt[n - 1] > W)
19         return knapsack(W, wt, val, n - 1);
20
21     else
22     {
23         return max(
24             val[n - 1] + knapsack(W - wt[n - 1], wt, val, n - 1),
25             knapsack(W, wt, val, n - 1));
26     }
27 }
28
29 int main()
30 {
31     int val[] = {60, 100, 120};
32     int wt[] = {10, 20, 30};
33
34     int W = 50;
35     int n = 3;
36
37     cout << "Maximum Profit = "
38          << knapsack(W, wt, val, n);
39
40     return 0;
41 }
```

```
DAA/prac [] main ?
> g++ knapsack.cpp -o knapsack

DAA/prac [] main ?
> ./knapsack
Maximum Profit = 220%
```